JAVA PROJECT REPORT ON

INVENTORY PURCHASE ORDER MANAGEMENT SYSTEM

Submitted by

VAISHNAVI V.D
25BIT059

Under the Guidance of

_____

Department of Information Technology
KG College of Arts and Science

Academic Year: 2026 – 2027
Semester: III
October 2026

KG COLLEGE OF ARTS AND SCIENCE

DECLARATION

I hereby declare that the project entitled **“Inventory Purchase Order Management System”** submitted in partial fulfillment of the requirements for the course **JAVA PROGRAMMING** is a record of my academic project work. The project demonstrates the use of Java programming concepts. Any external sources used for reference are acknowledged in the references section.

Date: _____

Place: Coimbatore

VAISHNAVI V.D
25BIT059

CERTIFICATE

This is to certify that the project entitled **“Inventory Purchase Order Management System”** is a project work carried out by **VAISHNAVI V.D (25BIT059)**, B.Sc. Information Technology, KG College of Arts and Science, in partial fulfillment of the requirements for the course **JAVA PROGRAMMING**. The project is submitted for academic evaluation under the guidance of the concerned faculty member.

Place: _____
Date: _____

Project Guide	Head of the Department
_____	_____

TABLE OF CONTENTS

S. No.	Topic	Page
1	Introduction	—
1.1	Objective	—
1.2	Overview	—
1.3	Features	—
2	Program Analysis	—
2.1	Problem Definition	—
2.2	Project Overview	—
2.3	Scope and Limitations	—
3	System Design	—
3.1	Modules	—
4	Methodology	—
4.1	Problem Analysis	—
4.2	System Design	—
4.3	Program Development	—
4.4	Testing and Validation	—
4.5	Result and Documentation	—
4.6	Algorithm	—
4.7	Flowchart	—
5	Source Code	—
6	Input and Output Screens	—
7	Discussion and Conclusion	—
7.1	Discussion	—
7.2	Conclusion	—
8	References	—

ABSTRACT

The Inventory Purchase Order Management System is a Java-based, menu-driven application that helps maintain basic inventory items and manage purchase orders. Manual inventory tracking may lead to stock-count errors, delayed purchasing, and difficulty identifying which items need replenishment. This application provides a structured way to view items, create purchase orders, receive ordered stock, and review order records.

The program contains sample inventory items with item IDs, names, unit prices, and stock quantities. A user can display the inventory, create a purchase order by selecting an item and quantity, calculate the order total, and receive the ordered quantity into stock. The system validates item IDs and quantities and updates stock after receipt. The project demonstrates Java classes, objects, ArrayList, methods, loops, conditional statements, switch-case, and input validation. This educational version stores information in memory only; database connectivity and supplier integration can be added later.

1. INTRODUCTION

Inventory management is important for shops, offices, warehouses, and organizations that purchase and store goods. Maintaining accurate stock information helps staff understand which products are available and when new supplies should be ordered. Purchase orders provide a record of the items and quantities requested from suppliers.

The Inventory Purchase Order Management System is designed to organize these basic activities in a simple Java application. It allows the user to view inventory, create a purchase order, list purchase orders, and receive stock against an order. The program calculates the purchase amount automatically and updates inventory when goods are received.

1.1 Objective

- Maintain a list of inventory items and their stock quantities.
- Display item IDs, names, prices, and available stock.
- Create purchase orders for selected items and quantities.
- Calculate the total purchase order value automatically.
- Generate an order ID and store order details.
- Mark orders as received and update stock quantities.
- Validate user inputs and demonstrate Core Java concepts.

1.2 Overview

The program starts with a menu. The user can view inventory, create a purchase order, view existing orders, receive an order, or exit. Each inventory item has an ID, name, unit price, and quantity in stock. Each purchase order records the order ID, item, quantity, total cost, and order status.

1.3 Features

- View inventory items
- Create purchase orders
- Automatic total-cost calculation
- Generate unique order IDs during the session
- View purchase order history
- Receive an order and update stock
- Basic input validation

2. PROGRAM ANALYSIS

Program analysis identifies the problems in manual inventory and purchasing and defines how the proposed application can help. The system is intended as a beginner-level academic project and does not replace a full enterprise inventory platform.

2.1 Problem Definition

Manual stock and purchase records can lead to inaccurate quantities, calculation mistakes, missing order details, and delays when checking which goods have been ordered. It can also be difficult to update stock consistently when new goods arrive.

The proposed system keeps inventory and purchase order records in Java collections while the program is running. It validates order quantities, calculates totals, and updates stock when an order is marked as received.

2.2 Project Overview

The application is a console-based Java program with a menu for inventory display, order creation, order listing, order receipt, and exit. The Event/Booking pattern is replaced with Item/PurchaseOrder classes suited to inventory purchasing.

Sample inventory data

Item ID	Item Name	Unit Price	Stock
I101	Notebook	■50.00	100
I102	Pen Pack	■80.00	60
I103	USB Drive	■450.00	25
I104	Keyboard	■650.00	15

2.3 Scope and Limitations

Scope: View inventory, create and review purchase orders, mark an order as received, and update stock quantities. The program is suitable for demonstrating object-oriented programming and collection handling.

Limitations: No database, supplier account, invoice printing, automatic low-stock alerts, authentication, or real-time multi-user support is included. Data is held in memory and is lost when the program closes.

3. SYSTEM DESIGN

The system uses a menu-driven structure and separates inventory and order data into classes. Each menu option calls a dedicated method. ArrayList collections store items and purchase orders during the current session.

3.1 Modules

1. Inventory Display Module: Displays item IDs, item names, unit prices, and current stock quantities.

2. Purchase Order Creation Module: Accepts an item ID and order quantity, checks the item, and creates an order record.

3. Cost Calculation Module: Calculates order value by multiplying the unit price by the requested quantity.

4. Order Listing Module: Displays purchase order IDs, item details, quantities, totals, and statuses.

5. Receive Order Module: Marks a pending order as received and increases stock by the order quantity.

6. Exit Module: Ends the program when selected.

System workflow

START
Load Sample Inventory
Display Main Menu
Read User Choice
View Inventory / Create Order / View Orders / Receive Order
Validate and Process the Selected Operation
Return to Menu or Exit
END

4. METHODOLOGY

The development process includes requirement analysis, system design, Java coding, testing, and documentation.

4.1 Problem Analysis

The main requirements are to maintain sample inventory, create purchase orders, calculate order totals, display order history, and update stock after receiving an order. Inputs must be validated to prevent unknown item IDs and invalid quantities.

4.2 System Design

The Item class stores item ID, name, unit price, and stock quantity. The PurchaseOrder class stores order ID, item, quantity, total cost, and received status. ArrayList stores item and order objects. Separate methods implement each menu operation.

4.3 Program Development

The project uses Java's Scanner class for keyboard input, ArrayList for storing objects, classes and constructors for data modeling, loops for menu repetition, and switch-case statements for menu navigation. Methods make the code easier to read and maintain.

4.4 Testing and Validation

Test Case	Expected Result
View inventory	All sample items and quantities appear.
Create a valid order	Order ID and total cost are displayed.
Enter unknown item ID	Error message; no order is created.
Enter zero/negative quantity	Invalid quantity message is displayed.
View purchase orders	Order history and status are displayed.
Receive a pending order	Order status changes and stock increases once.
Receive the same order again	System reports that it has already been received.

4.5 Result and Documentation

The program is designed to demonstrate inventory display, purchase order creation, and stock receipt. Run the program and capture screenshots from your own Java environment if required. Example console outputs in this report are illustrative and should be verified against the actual run.

4.6 ALGORITHM

1. Start the program.
2. Load the sample inventory list and create an empty purchase order list.
3. Display the main menu.
4. Read the user's menu choice.
5. If View Inventory is selected, display all items and stock quantities.
6. If Create Purchase Order is selected, read item ID and quantity.
7. Validate the item ID and ensure the quantity is positive.
8. Calculate the total amount and create a purchase order ID.
9. If View Orders is selected, display the order records and statuses.
10. If Receive Order is selected, find a pending order, mark it received, and add its quantity to stock.
11. Repeat the menu until Exit is selected.
12. End the program.

4.7 FLOWCHART

START
Initialize Inventory and Order List
Display Main Menu
Read Choice and Perform Operation
Return to Menu? — Yes
No / Exit → END

5. SOURCE CODE

Save the following code as **InventoryPurchaseOrderSystem.java**. The Java file name must match the public class name. The application uses sample inventory records and keeps purchase orders in memory for the current session.

```
import java.util.ArrayList;
import java.util.Scanner;

class Item {
    String id;
    String name;
    double price;
    int stock;

    Item(String id, String name, double price, int stock) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.stock = stock;
    }
}

class PurchaseOrder {
    String orderId;
    Item item;
    int quantity;
    double total;
    boolean received;

    PurchaseOrder(String orderId, Item item, int quantity) {
        this.orderId = orderId;
        this.item = item;
        this.quantity = quantity;
        this.total = item.price * quantity;
        this.received = false;
    }
}

public class InventoryPurchaseOrderSystem {
    private static final Scanner sc = new Scanner(System.in);
    private static final ArrayList<Item> inventory = new ArrayList<>();
    private static final ArrayList<PurchaseOrder> orders = new ArrayList<>();
    private static int nextOrderId = 1001;

    public static void main(String[] args) {
        loadInventory();
        int choice;

        do {
            System.out.println("\\n====================================");
            System.out.println(" INVENTORY PURCHASE ORDER MANAGEMENT");
            System.out.println(" Student: Vaishnavi V.D | 25BIT059");
            System.out.println("====================================");
            System.out.println("1. View Inventory");
            System.out.println("2. Create Purchase Order");
            System.out.println("3. View Purchase Orders");
            System.out.println("4. Receive Purchase Order");
```

```
        System.out.println("5. Exit");
        System.out.print("Enter your choice: ");
        choice = readInt();

        switch (choice) {
            case 1: displayInventory(); break;
            case 2: createOrder(); break;
            case 3: viewOrders(); break;
            case 4: receiveOrder(); break;
            case 5: System.out.println("Thank you for using the system!"); break;
            default: System.out.println("Invalid choice. Try again.");
        }
    } while (choice != 5);

    sc.close();
}

private static void loadInventory() {
    inventory.add(new Item("I101", "Notebook", 50.00, 100));
    inventory.add(new Item("I102", "Pen Pack", 80.00, 60));
    inventory.add(new Item("I103", "USB Drive", 450.00, 25));
    inventory.add(new Item("I104", "Keyboard", 650.00, 15));
}

private static void displayInventory() {
    System.out.println("\\nCURRENT INVENTORY");
    System.out.printf("%-8s %-18s %-12s %-8s%n",
            "Item ID", "Item Name", "Unit Price", "Stock");
    for (Item item : inventory) {
        System.out.printf("%-8s %-18s Rs.%-9.2f %-8d%n",
                item.id, item.name, item.price, item.stock);
    }
}

private static Item findItem(String id) {
    for (Item item : inventory) {
        if (item.id.equalsIgnoreCase(id)) return item;
    }
    return null;
}

private static PurchaseOrder findOrder(String id) {
    for (PurchaseOrder order : orders) {
        if (order.orderId.equalsIgnoreCase(id)) return order;
    }
    return null;
}

private static void createOrder() {
    displayInventory();
    System.out.print("Enter item ID: ");
    String itemId = sc.nextLine().trim();
```

```
        Item item = findItem(itemId);

        if (item == null) {
            System.out.println("Item ID not found.");
            return;
        }

        System.out.print("Enter order quantity: ");
        int quantity = readInt();
        if (quantity <= 0) {
            System.out.println("Quantity must be greater than zero.");
            return;
        }

        PurchaseOrder order = new PurchaseOrder(
                "PO" + nextOrderId++, item, quantity);
        orders.add(order);

        System.out.println("\\nPURCHASE ORDER CREATED");
        System.out.println("Order ID   : " + order.orderId);
        System.out.println("Item       : " + item.name);
        System.out.println("Quantity   : " + quantity);
        System.out.printf("Unit Price : Rs. %.2f%n", item.price);
        System.out.printf("Total      : Rs. %.2f%n", order.total);
        System.out.println("Status     : PENDING RECEIPT");
    }

    private static void viewOrders() {
        System.out.println("\\nPURCHASE ORDER HISTORY");
        if (orders.isEmpty()) {
            System.out.println("No purchase orders found.");
            return;
        }

        for (PurchaseOrder order : orders) {
            System.out.printf("%s | %s | Qty: %d | Total: Rs. %.2f | %s%n",
                    order.orderId, order.item.name, order.quantity,
                    order.total, order.received ? "RECEIVED" : "PENDING");
        }
    }

    private static void receiveOrder() {
        System.out.print("Enter purchase order ID: ");
        String id = sc.nextLine().trim();
        PurchaseOrder order = findOrder(id);

        if (order == null) {
            System.out.println("Purchase order not found.");
            return;
        }
        if (order.received) {
            System.out.println("This order has already been received.");
```

```
            return;
        }

        order.received = true;
        order.item.stock += order.quantity;
        System.out.println("Order " + order.orderId
                + " marked as received.");
        System.out.println("Inventory stock updated successfully.");
    }

    private static int readInt() {
        while (!sc.hasNextInt()) {
            System.out.print("Please enter a whole number: ");
            sc.nextLine();
        }
        int value = sc.nextInt();
        sc.nextLine();
        return value;
    }
}
```

6. INPUT AND OUTPUT SCREENS

Run the Java program and capture actual screenshots from your console if required by your faculty. These sample outputs show the expected style of results.

Figure 1: Main Menu

```
=====================================
 INVENTORY PURCHASE ORDER MANAGEMENT
 Student: Vaishnavi V.D | 25BIT059
=====================================
1. View Inventory
2. Create Purchase Order
3. View Purchase Orders
4. Receive Purchase Order
5. Exit
Enter your choice: 1
```

Figure 2: Inventory List

```
CURRENT INVENTORY
Item ID  Item Name          Unit Price   Stock
I101     Notebook           Rs.50.00     100
I102     Pen Pack           Rs.80.00     60
I103     USB Drive          Rs.450.00    25
I104     Keyboard           Rs.650.00    15
```

Figure 3: Purchase Order Created (example)

```
Enter item ID: I101
Enter order quantity: 20

PURCHASE ORDER CREATED
Order ID   : PO1001
Item       : Notebook
Quantity   : 20
Unit Price : Rs. 50.00
Total      : Rs. 1000.00
Status     : PENDING RECEIPT
```

Figure 4: Order Received (example)

```
Enter purchase order ID: PO1001
Order PO1001 marked as received.
Inventory stock updated successfully.
```

7. DISCUSSION AND CONCLUSION

7.1 Discussion

The Inventory Purchase Order Management System demonstrates how Java can be used to organize inventory data and purchasing records. The Item and PurchaseOrder classes represent the main entities, while ArrayList collections store their objects. The menu provides operations to view inventory, create orders, view order history, and receive goods.

The application calculates order totals automatically and updates stock when a purchase order is received. It checks invalid item IDs, invalid quantities, missing orders, and attempts to receive the same order more than once. Since the application is an academic prototype, it does not include supplier communication or permanent data storage.

7.2 Conclusion

The project provides a simple Java-based approach to basic inventory and purchase order management. It helps demonstrate object-oriented programming, constructors, methods, collections, loops, switch-case, and validation. The project can be enhanced by adding database connectivity using JDBC, supplier details, low-stock alerts, purchase order reports, login security, and a graphical user interface.

8. REFERENCES

Books

• Herbert Schildt, *Java: The Complete Reference*, McGraw-Hill.
• Cay S. Horstmann, *Core Java Volume I: Fundamentals*, Pearson Education.
• Y. Daniel Liang, *Introduction to Java Programming*, Pearson Education.

Websites

• https://docs.oracle.com/en/java/
• https://www.geeksforgeeks.org/java/
• https://www.w3schools.com/java/

— END OF PROJECT REPORT —